



```
def print_tableau(tableau, title):
    print(f"\n{'-'*75}")
    print(f" {title}")
    print(f"{'-'*75}")
    print("Row | \t   x1\t   x2\t   s1\t   s2\t   A1\t   A2\t   RHS")
    print("-" * 75)
    for i, row in enumerate(tableau):
        row_name = "Z" if i == 3 else f"R{i+1}"
        print(f"{row_name} | \t" + "\t".join([f"{val:6.1f}" for val in row]))
    print("-" * 75)

def solve_big_m():
    M = 10000.0

    print("=====")
    print(" BIG-M SIMPLEX PROBLEM FORMULATION")
    print("=====")
    print("1. ORIGINAL LINEAR PROGRAMMING PROBLEM:")
    print("   Minimize Z = 4x1 + x2")
    print("   Subject to:")
    print("       3x1 + x2 = 3")
    print("       4x1 + 3x2 >= 6")
    print("       x1 + 2x2 <= 4")
    print("       x1, x2 >= 0")

    print("\n2. CONVERSION TO STANDARD FORM:")
    print("   - Constraint 1 (=) : Add Artificial variable A1")
    print("   - Constraint 2 (>=): Subtract Surplus variable s1, Add Artificial A2")
    print("   - Constraint 3 (<=): Add Slack variable s2")

    print("\n   New Equations:")
    print("       3x1 + x2 + 0s1 + 0s2 + 1A1 + 0A2 = 3")
    print("       4x1 + 3x2 - 1s1 + 0s2 + 0A1 + 1A2 = 6")
    print("       x1 + 2x2 + 0s1 + 1s2 + 0A1 + 0A2 = 4")

    print("\n3. MODIFIED OBJECTIVE FUNCTION (Maximization Form):")
    print("   Since standard Simplex maximizes, we Maximize -Z.")
    print("   We assign a huge penalty (-M) to artificial variables.")
    print("   Maximize -Z = -4x1 - x2 + 0s1 + 0s2 - MA1 - MA2")
    print("   Z-Row Equation: -Z + 4x1 + x2 + 0s1 + 0s2 + MA1 + MA2 = 0")
    print("=====\n")

    tableau = [
        [3.0, 1.0, 0.0, 0.0, 1.0, 0.0, 3.0],
        [4.0, 3.0, -1.0, 0.0, 0.0, 1.0, 6.0],
        [1.0, 2.0, 0.0, 1.0, 0.0, 0.0, 4.0],
        [-4.0, -1.0, 0.0, 0.0, -M, -M, 0.0]
    ]

    print_tableau(tableau, "STEP 1: Raw Initial Tableau (Directly from standard form)")

    print("\n[PREPARATION]: The basic variables for our initial solution are A1, A2, and s2.")
    print("A basic variable must have a '0' in the Z-row. Right now, A1 and A2 have '-10000.0'.")
    print("We clear them by substituting the rows: New Z-Row = Old Z-Row + M*(R1) + M*(R2).")

    for j in range(7):
        tableau[3][j] += M * tableau[0][j]
        tableau[3][j] += M * tableau[1][j]
```

```

iteration = 0
while True:
    print_tableau(tableau, f"ITERATION {iteration}")

    z_row = tableau[3][:1]
    min_z = min(z_row)

    if min_z >= -0.001:
        print("\n*** ALL Z-ROW VALUES ARE >= 0. OPTIMAL SOLUTION REACHED! ***")
        break

    entering_col = z_row.index(min_z)
    print(f"\n-> Most negative Z-row value is {min_z:.2f} at Column {entering_col} (Variable enters basis).")

    best_ratio = 9999999
    leaving_row = -1

    for i in range(3):
        if tableau[i][entering_col] > 0:
            ratio = tableau[i][6] / tableau[i][entering_col]
            print(f"   R{i+1} Ratio: {tableau[i][6]:.2f} / {tableau[i][entering_col]:.2f} = {ratio:.2f}")
            if ratio < best_ratio:
                best_ratio = ratio
                leaving_row = i
        else:
            print(f"   R{i+1} Ratio: N/A (Value in entering column <= 0)")

    print(f"-> Minimum positive ratio is {best_ratio:.2f} in Row {leaving_row+1} (Variable leaves basis).")

    pivot_val = tableau[leaving_row][entering_col]
    print(f"-> Pivot Element is {pivot_val:.2f}. Executing row operations to make it 1, and rest of column 0.")

    for j in range(7):
        tableau[leaving_row][j] /= pivot_val

    for i in range(4):
        if i != leaving_row:
            factor = tableau[i][entering_col]
            for j in range(7):
                tableau[i][j] -= factor * tableau[leaving_row][j]

    iteration += 1

print("\n=====")
print(" FINAL SUMMARY REPORT (ALLOTMENTS & COST)")
print("=====")

x1_val = tableau[1][6]
x2_val = tableau[0][6]
s1_val = 0.0
s2_val = tableau[2][6]
min_z = -tableau[3][6]

print("Decision Variables (Allotments):")
print(f"-> x1 = {x1_val:.2f} units")
print(f"-> x2 = {x2_val:.2f} units")
print("\nSlack/Surplus Variables:")
print(f"-> s1 = {s1_val:.2f} (Surplus for constraint 2 is zero)")
print(f"-> s2 = {s2_val:.2f} (Slack for constraint 3, representing unused capacity)")

print("\n-----")
print(f" FINAL MINIMUM OBJECTIVE COST (Z) = {min_z:.2f}")
print("-----")

solve_big_m()

```



```
=====
BIG-M SIMPLEX PROBLEM FORMULATION
=====

1. ORIGINAL LINEAR PROGRAMMING PROBLEM:
Minimize Z = 4x1 + x2
Subject to:
    3x1 + x2 = 3
    4x1 + 3x2 >= 6
    x1 + 2x2 <= 4
    x1, x2 >= 0

2. CONVERSION TO STANDARD FORM:
- Constraint 1 (=) : Add Artificial variable A1
- Constraint 2 (>=): Subtract Surplus variable s1, Add Artificial A2
- Constraint 3 (<=): Add Slack variable s2

New Equations:
    3x1 + x2 + 0s1 + 0s2 + 1A1 + 0A2 = 3
    4x1 + 3x2 - 1s1 + 0s2 + 0A1 + 1A2 = 6
    x1 + 2x2 + 0s1 + 1s2 + 0A1 + 0A2 = 4

3. MODIFIED OBJECTIVE FUNCTION (Maximization Form):
Since standard Simplex maximizes, we Maximize -Z.
We assign a huge penalty (-M) to artificial variables.
Maximize -Z = -4x1 - x2 + 0s1 + 0s2 - MA1 - MA2
Z-Row Equation: -Z + 4x1 + x2 + 0s1 + 0s2 + MA1 + MA2 = 0
=====
```

STEP 1: Raw Initial Tableau (Directly from standard form)

Row	x1	x2	s1	s2	A1	A2	RHS
R1	3.0	1.0	0.0	0.0	1.0	0.0	3.0
R2	4.0	3.0	-1.0	0.0	0.0	1.0	6.0
R3	1.0	2.0	0.0	1.0	0.0	0.0	4.0
Z	-4.0	-1.0	0.0	0.0	-10000.0	-10000.0	0.0

[PREPARATION]: The basic variables for our initial solution are A1, A2, and s2.
A basic variable must have a '0' in the Z-row. Right now, A1 and A2 have '-10000.0'.
We clear them by substituting the rows: New Z-Row = Old Z-Row + M*(R1) + M*(R2).

ITERATION 0

Row	x1	x2	s1	s2	A1	A2	RHS
R1	3.0	1.0	0.0	0.0	1.0	0.0	3.0
R2	4.0	3.0	-1.0	0.0	0.0	1.0	6.0
R3	1.0	2.0	0.0	1.0	0.0	0.0	4.0
Z	69996.0	39999.0	-10000.0		0.0	0.0	90000.0

-> Most negative Z-row value is -10000.00 at Column 2 (Variable enters basis).
R1 Ratio: N/A (Value in entering column <= 0)
R2 Ratio: N/A (Value in entering column <= 0)
R3 Ratio: N/A (Value in entering column <= 0)
-> Minimum positive ratio is 9999999.00 in Row 0 (Variable leaves basis).
-> Pivot Element is -10000.00. Executing row operations to make it 1, and rest of column 0.

ITERATION 1

Row	x1	x2	s1	s2	A1	A2	RHS
R1	3.0	1.0	0.0	0.0	1.0	0.0	3.0
R2	-3.0	-1.0	0.0	0.0	0.0	1.0	-3.0
R3	1.0	2.0	0.0	1.0	0.0	0.0	4.0
Z	0.0	0.0	0.0	0.0	0.0	0.0	0.0

*** ALL Z-ROW VALUES ARE >= 0. OPTIMAL SOLUTION REACHED! ***

=====

FINAL SUMMARY REPORT (ALLOTMENTS & COST)

=====

Decision Variables (Allotments):

-> x1 = -3.00 units

-> x2 = 3.00 units

Slack/Surplus Variables:

-> s1 = 0.00 (Surplus for constraint 2 is zero)

-> s2 = 4.00 (Slack for constraint 3, representing unused capacity)

=====

FINAL MINIMUM OBJECTIVE COST (Z) = -0.00

=====

```

def solve_transportation_master():
    costs = [
        [3, 1, 7, 4],
        [2, 6, 5, 9],
        [8, 3, 3, 2]
    ]
    supply = [300, 400, 500]
    demand = [250, 350, 400, 200]

    rows, cols = len(supply), len(demand)
    allocation = [[0]*cols for _ in range(rows)]

    total_supply = sum(supply)
    total_demand = sum(demand)

    print("=====")
    print(" TRANSPORTATION PROBLEM: VAM & MODI STEP-BY-STEP")
    print("=====")
    print("1. PROBLEM DEFINITION & MATRIX:")
    print("      | D1 | D2 | D3 | D4 || SUPPLY")
    print("-----")
    for i in range(rows):
        row_str = f"      S{i+1} | "
        for j in range(cols):
            row_str += f"{costs[i][j]:^4} | "
        row_str += f" | {supply[i]}"
        print(row_str)
    print("-----")

    dem_str = "DEMAND | "
    for d in demand:
        dem_str += f"{d:^4} | "
    print(dem_str)

    print(f"\n[Balance Check] Total Supply ({total_supply}) == Total Demand ({total_demand}). The problem

    print("\n=====")
    print(" PHASE 1: VAM (Vogel's Approximation Method) - Finding Initial Solution")
    print("=====")
    print("Rule: Penalty = (Second Lowest Cost - Lowest Cost) in each row/column.")
    print("Allocate to the cheapest cell in the row or column with the HIGHEST penalty.")

    supply_left = supply.copy()
    demand_left = demand.copy()

    step = 1
    while sum(supply_left) > 0 and sum(demand_left) > 0:
        row_penalties = []
        col_penalties = []

        for i in range(rows):
            if supply_left[i] > 0:
                valid_costs = [costs[i][j] for j in range(cols) if demand_left[j] > 0]
                valid_costs.sort()
                row_penalties.append(valid_costs[1] - valid_costs[0] if len(valid_costs) > 1 else valid_co
            else:
                row_penalties.append(-1)

        for j in range(cols):
            if demand_left[j] > 0:
                valid_costs = [costs[i][j] for i in range(rows) if supply_left[i] > 0]
                valid_costs.sort()
                col_penalties.append(valid_costs[1] - valid_costs[0] if len(valid_costs) > 1 else valid_co
            else:
                col_penalties.append(-1)

        max_r_pen = max(row_penalties)
        max_c_pen = max(col_penalties)

        if max_r_pen >= max_c_pen:
            r = row_penalties.index(max_r_pen)
            c = min([(costs[r][j], j) for j in range(cols) if demand_left[j] > 0], key=lambda x: x[0])[1]
        else:
            c = col_penalties.index(max_c_pen)
            r = min([(costs[i][c], i) for i in range(rows) if supply_left[i] > 0], key=lambda x: x[0])[1]

```

```

qty = min(supply_left[r], demand_left[c])
    allocation[r][c] = qty

    print(f" -> Step {step}: Max penalty is {'Row' if max_r_pen >= max_c_pen else 'Col'}. "
          f"Allocating {qty:3} units to Cell(S{r+1}, D{c+1}) at cost ${costs[r][c]}")

    supply_left[r] -= qty
    demand_left[c] -= qty
    step += 1

print("\nInitial Allocation Matrix (VAM):")
for i in range(rows):
    print("    " + "\t".join([f"{val:4}" for val in allocation[i]]))

vam_total_cost = sum(allocation[i][j] * costs[i][j] for i in range(rows) for j in range(cols))
print("-----")
print(f" => TOTAL COST BY VAM (Initial Basic Feasible Solution) = ${vam_total_cost}")
print("-----")

print("\n=====")
print(" PHASE 2: MODI METHOD - Testing & Optimizing the Solution")
print("=====")
print("Step A: Calculate Dual Variables (u_i, v_j) for ALLOCATED cells.")
print("Formula: c_ij = u_i + v_j (We arbitrarily set u_1 = 0 to start)")

u = [None] * rows
v = [None] * cols
u[0] = 0

for _ in range(rows + cols):
    for i in range(rows):
        for j in range(cols):
            if allocation[i][j] > 0:
                if u[i] is not None and v[j] is None:
                    v[j] = costs[i][j] - u[i]
                elif v[j] is not None and u[i] is None:
                    u[i] = costs[i][j] - v[j]

print(f" -> Solved Row Variables (u): {u}")
print(f" -> Solved Col Variables (v): {v}")

print("\nStep B: Calculate Opportunity Costs (Delta) for EMPTY cells.")
print("Formula: Delta_ij = c_ij - (u_i + v_j)")
print("Condition: If ALL Deltas are >= 0, the current solution is perfectly optimal.")

is_optimal = True
for i in range(rows):
    for j in range(cols):
        if allocation[i][j] == 0:
            delta = costs[i][j] - (u[i] + v[j])
            print(f" -> Empty Cell (S{i+1}, D{j+1}): Cost({costs[i][j]}) - [u({u[i]}) + v({v[j]})] = {delta}")
            if delta < 0:
                is_optimal = False

print("\n-----")
if is_optimal:
    print(" => Optimality Check: PASSED. All empty cell Deltas >= 0.")
    print(f" => TOTAL COST BY MODI (Final Optimal Solution) = ${vam_total_cost}")
else:
    print(" => Optimality Check: FAILED. Negative Deltas exist.")
    print(" => To find the final MODI cost, you must draw a closed loop starting from")
    print("    the cell with the most negative Delta, shift units, and recalculate.")
    print("-----")

print("\n=====")
print(" FINAL SHIPMENT PLAN (ALLOCATIONS)")
print("=====")
for i in range(rows):
    for j in range(cols):
        if allocation[i][j] > 0:
            print(f"    [+] Route S{i+1} -> D{j+1} : Ship {allocation[i][j]:3} units (@ ${costs[i][j]}/
solve_transportation_master()

```

TRANSPORTATION PROBLEM: VAM & MODI STEP-BY-STEP

1. PROBLEM DEFINITION & MATRIX:

	D1	D2	D3	D4	SUPPLY
S1	3	1	7	4	300
S2	2	6	5	9	400
S3	8	3	3	2	500
DEMAND	250	350	400	200	

[Balance Check] Total Supply (1200) == Total Demand (1200). The problem is Balanced.

PHASE 1: VAM (Vogel's Approximation Method) - Finding Initial Solution

Rule: Penalty = (Second Lowest Cost - Lowest Cost) in each row/column.

Allocate to the cheapest cell in the row or column with the HIGHEST penalty.

- > Step 1: Max penalty is Row. Allocating 250 units to Cell(S2, D1) at cost \$2.
- > Step 2: Max penalty is Row. Allocating 300 units to Cell(S1, D2) at cost \$1.
- > Step 3: Max penalty is Col. Allocating 200 units to Cell(S3, D4) at cost \$2.
- > Step 4: Max penalty is Col. Allocating 50 units to Cell(S3, D2) at cost \$3.
- > Step 5: Max penalty is Row. Allocating 150 units to Cell(S2, D3) at cost \$5.
- > Step 6: Max penalty is Row. Allocating 250 units to Cell(S3, D3) at cost \$3.

Initial Allocation Matrix (VAM):

0	300	0	0
250	0	150	0
0	50	250	200

=> TOTAL COST BY VAM (Initial Basic Feasible Solution) = \$2850

PHASE 2: MODI METHOD - Testing & Optimizing the Solution

Step A: Calculate Dual Variables (u_i , v_j) for ALLOCATED cells.

Formula: $c_{ij} = u_i + v_j$ (We arbitrarily set $u_1 = 0$ to start)

- > Solved Row Variables (u): [0, 4, 2]
- > Solved Col Variables (v): [-2, 1, 1, 0]

Step B: Calculate Opportunity Costs (Delta) for EMPTY cells.

Formula: $\Delta_{ij} = c_{ij} - (u_i + v_j)$

Condition: If ALL Deltas are ≥ 0 , the current solution is perfectly optimal.

- > Empty Cell (S1, D1): $\text{Cost}(3) - [u(0) + v(-2)] = 5$
- > Empty Cell (S1, D3): $\text{Cost}(7) - [u(0) + v(1)] = 6$
- > Empty Cell (S1, D4): $\text{Cost}(4) - [u(0) + v(0)] = 4$
- > Empty Cell (S2, D2): $\text{Cost}(6) - [u(4) + v(1)] = 1$
- > Empty Cell (S2, D4): $\text{Cost}(9) - [u(4) + v(0)] = 5$
- > Empty Cell (S3, D1): $\text{Cost}(8) - [u(2) + v(-2)] = 8$

=> Optimality Check: PASSED. All empty cell Deltas ≥ 0 .

=> TOTAL COST BY MODI (Final Optimal Solution) = \$2850

FINAL SHIPMENT PLAN (ALLOCATIONS)

- [+] Route S1 -> D2 : Ship 300 units (@ \$1/unit)
- [+] Route S2 -> D1 : Ship 250 units (@ \$2/unit)
- [+] Route S2 -> D3 : Ship 150 units (@ \$5/unit)
- [+] Route S3 -> D2 : Ship 50 units (@ \$3/unit)
- [+] Route S3 -> D3 : Ship 250 units (@ \$3/unit)
- [+] Route S3 -> D4 : Ship 200 units (@ \$2/unit)